

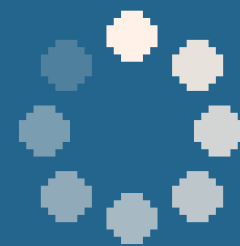
DESAFIO 1:

PERCEPTRON PARA PROBLEMAS **NÃO** LINEARES

Start

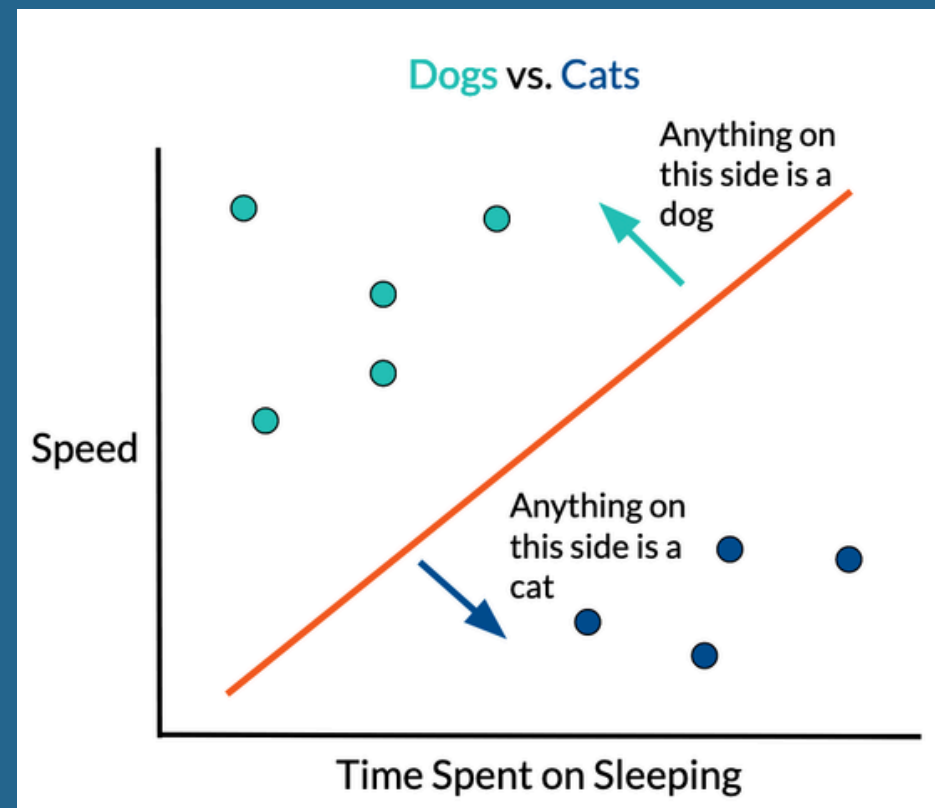


O QUE É O PERCEPTRON?

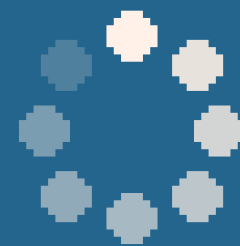


O QUE É O PERCEPTRON?

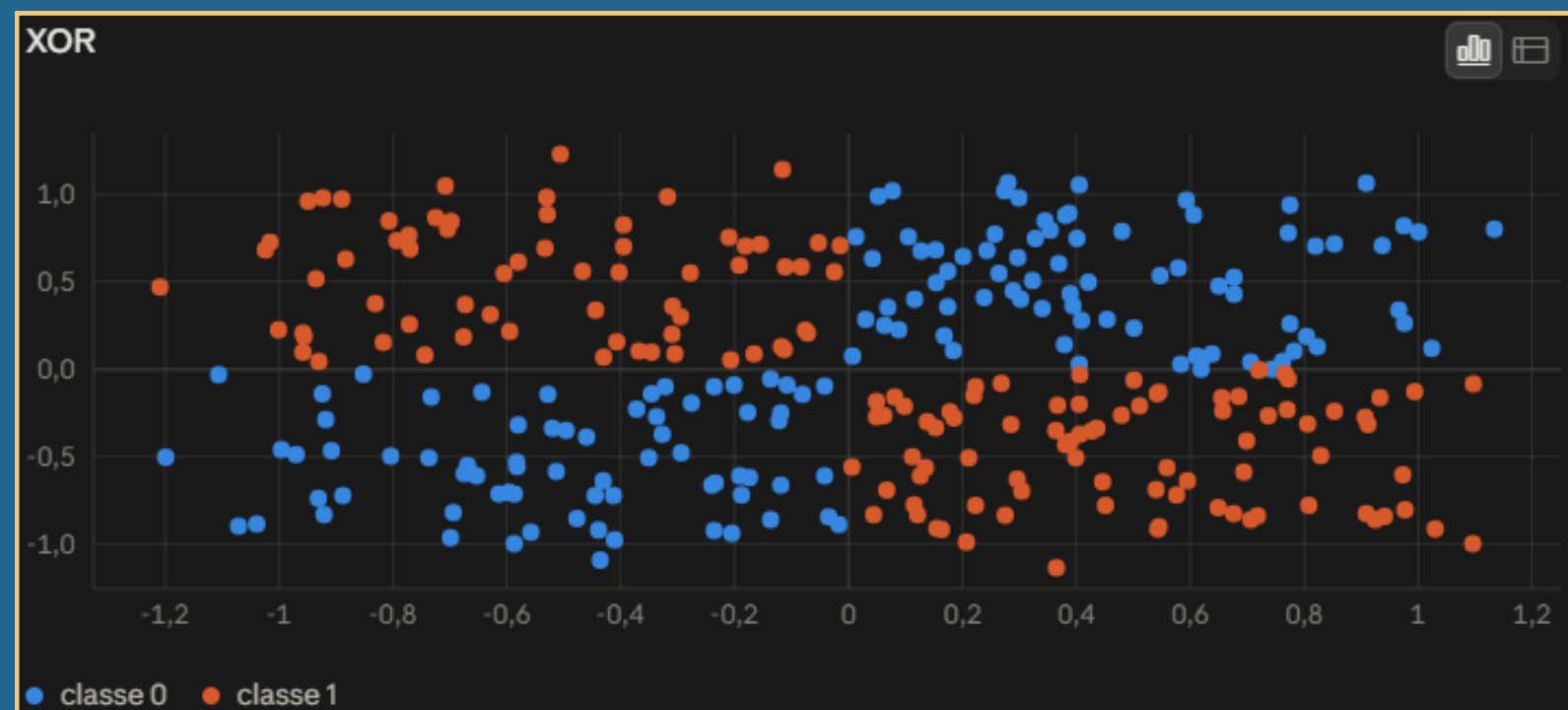
- Divide pontos de **duas classes diferentes** em **dois lados com uma reta**
- **Neurônio** = Menor “unidade biológica de inteligência”



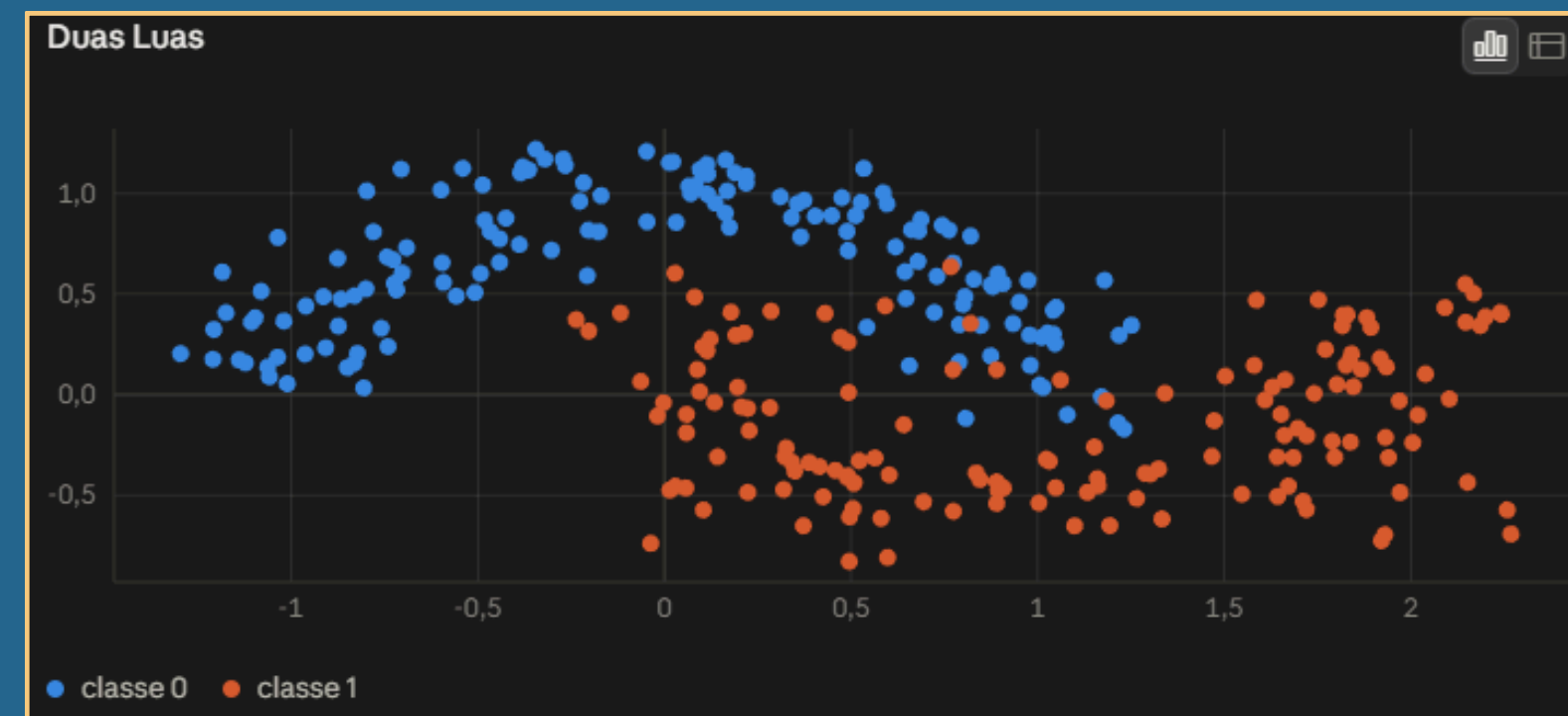
QUAL FOI O **DESAFIO** PROPOSTO?



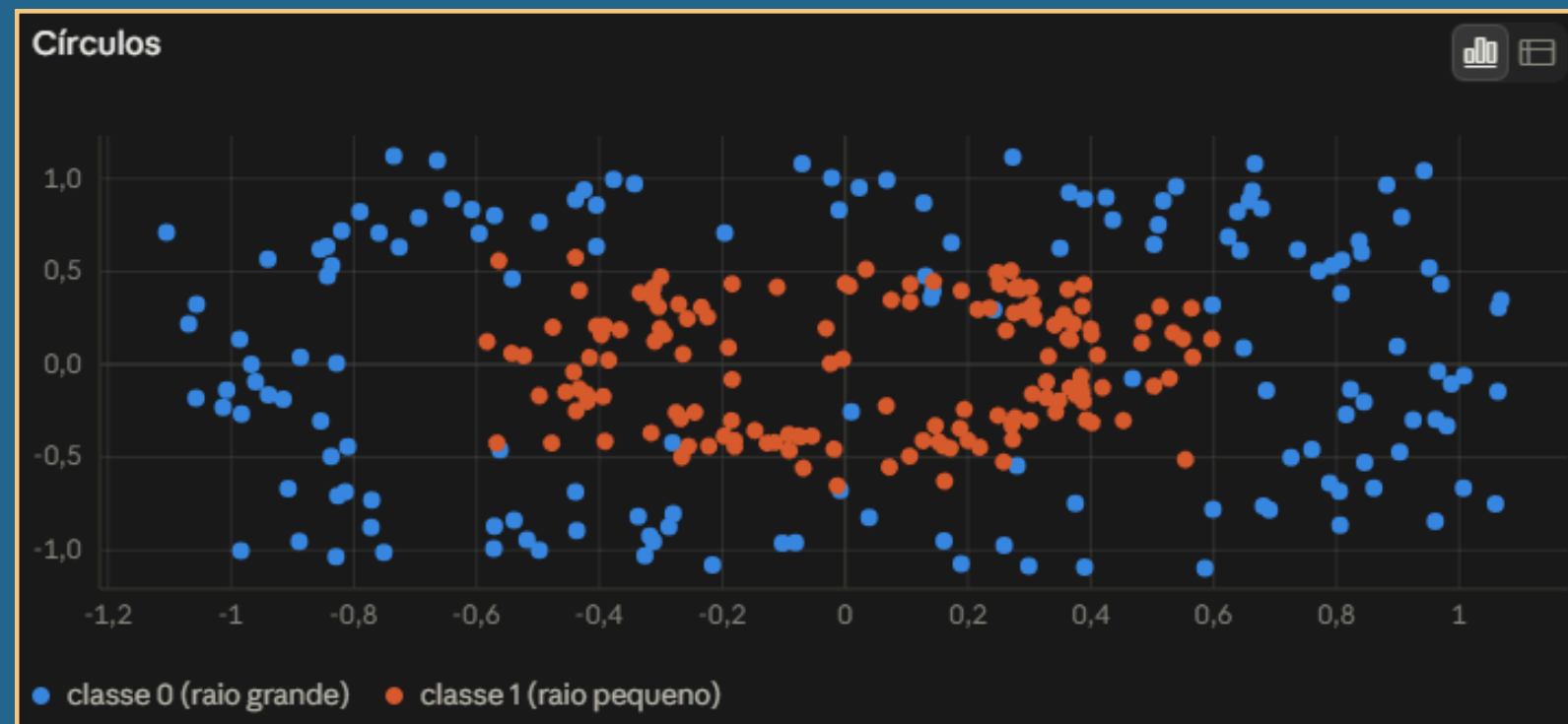
XOR



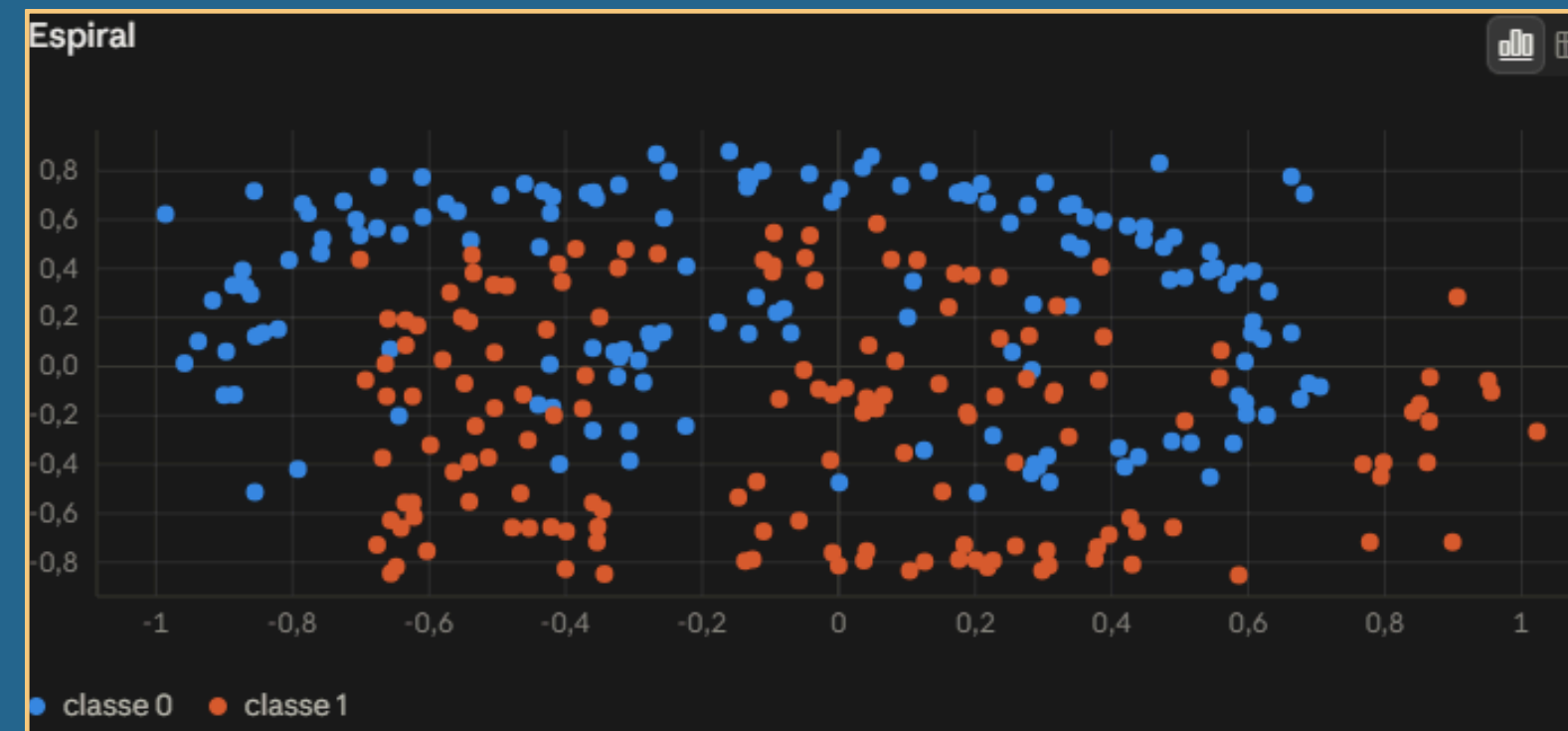
DUAS LUAS



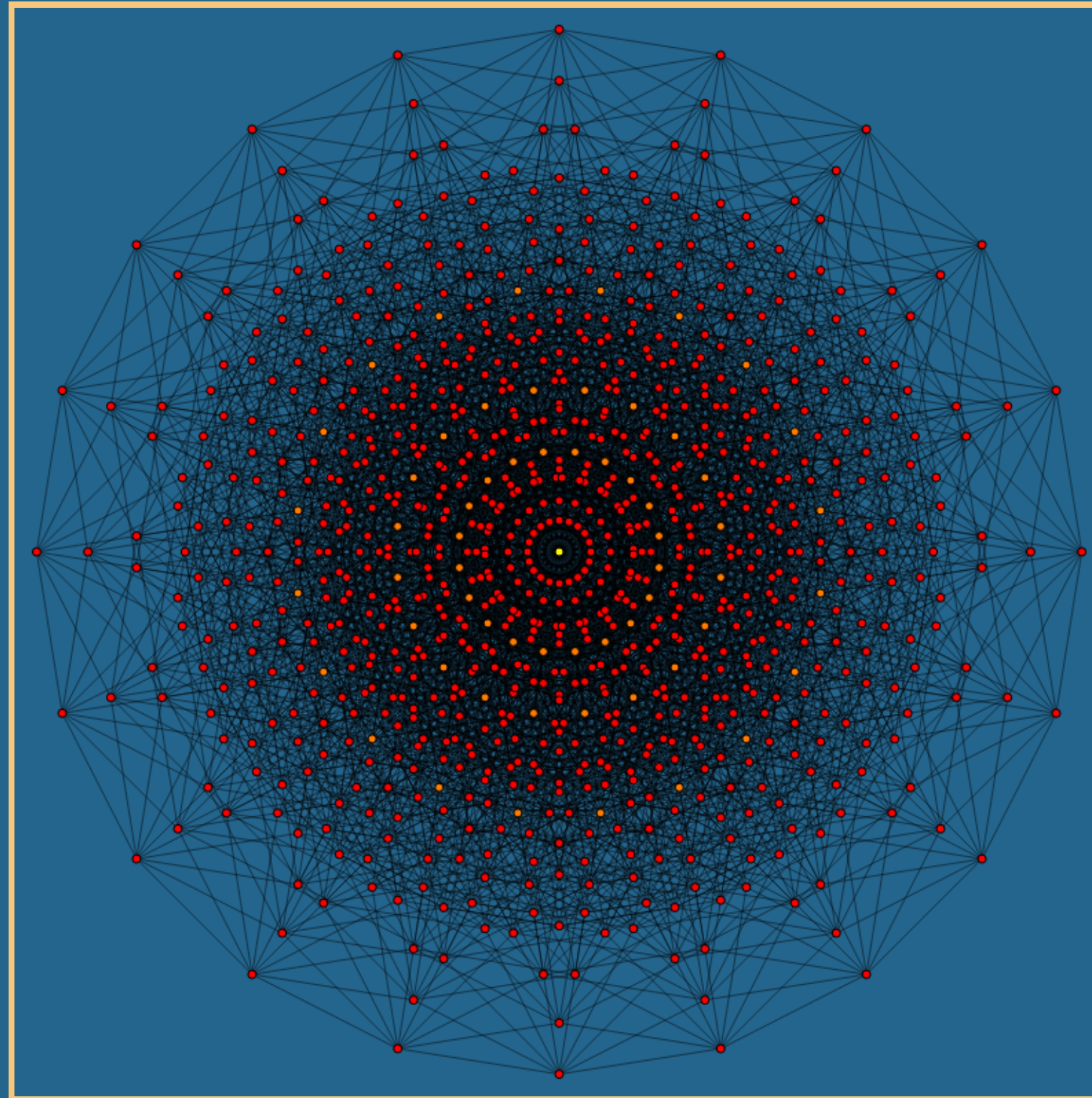
CÍRCULOS



ESPIRAL



ESFERA DE 10 DIMENSÕES



COMO FOI FEITO?



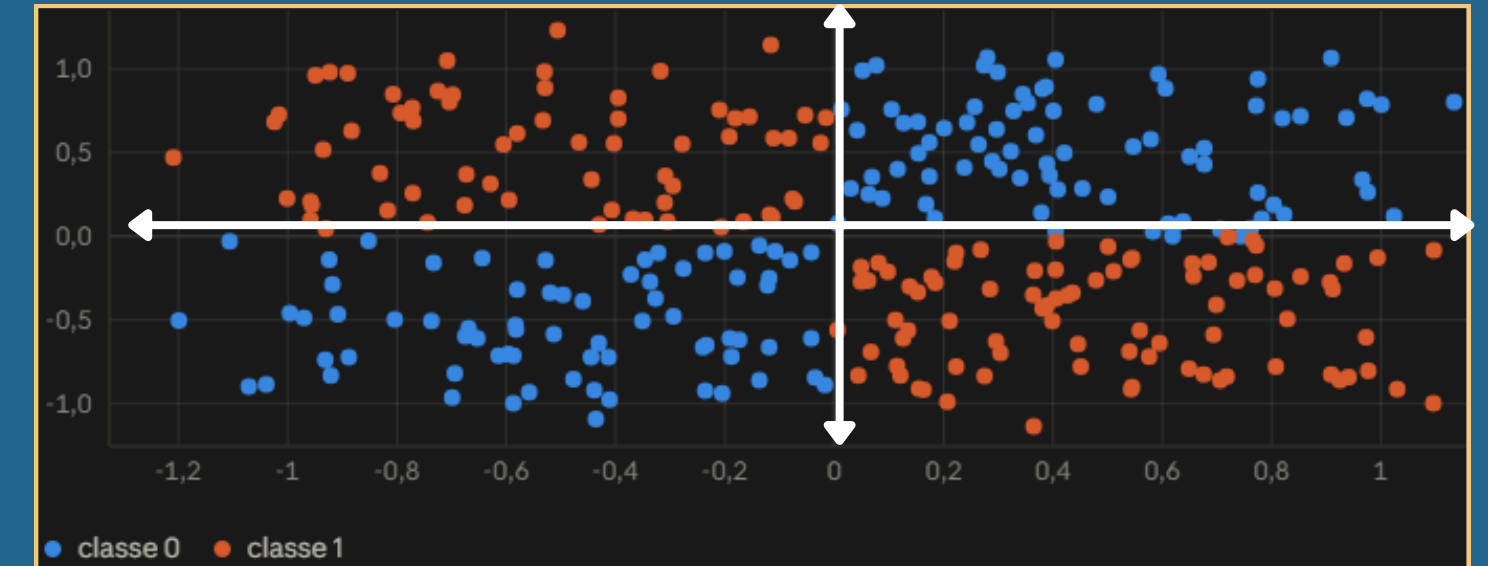
COMO FOI FEITO?

```
def fit(self, X: torch.Tensor) -> None:

    self.mu, self.sd = X.mean(0), X.std(0) + 1e-8
    d = X.shape[1]
    Xs = (X - self.mu) / self.sd
```

FIT - NORMALIZAÇÃO DOS VALORES

- Recebe um par de valores: média e média ponderada
- Recebe a dimensão espacial dos valores
- Deixa cada coluna com média 0 e "escala" 1, pra nenhuma variável dominar as outras só por ter números maiores.

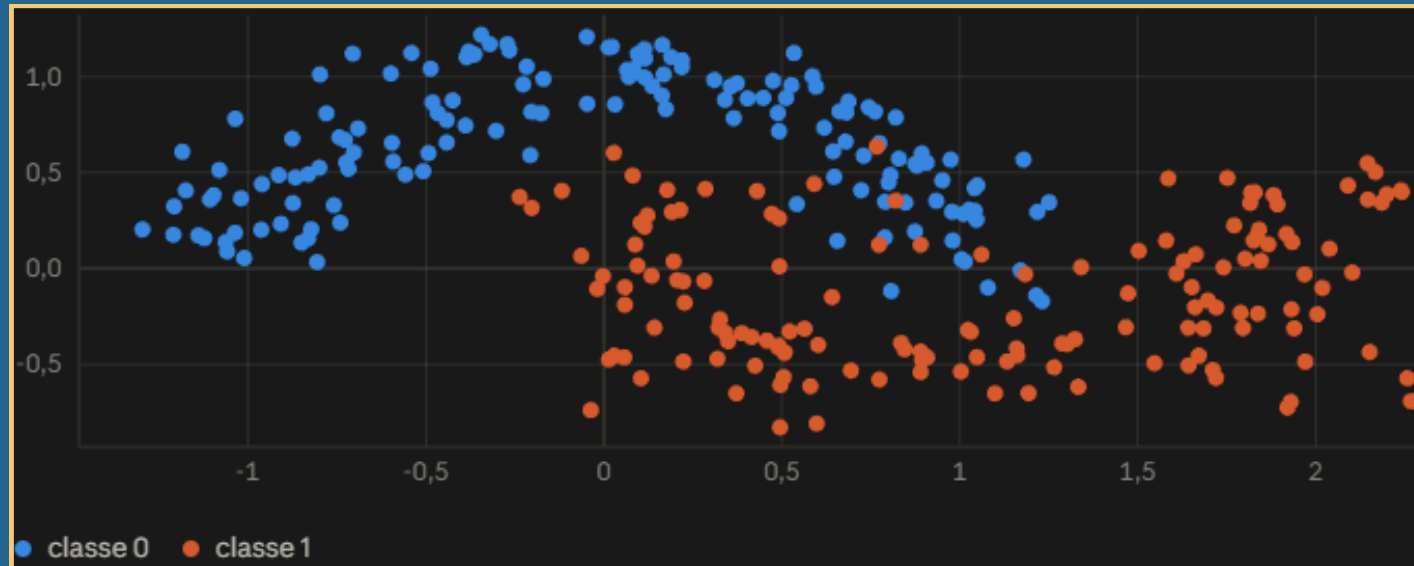


XOR - PRODUTOS CRUZADOS

- x e y tem sinais iguais? **Classe 0**
- x e y tem sinais diferentes? **Classe 1**
- Calcula-se $x*y$ e o Perceptron aprende a **separar o resultado do produto**

```
if self.pares:
    prod = torch.stack([Xs[:, i] * Xs[:, j] for i, j in self.pares], dim=1)
    partes.append(prod)
```

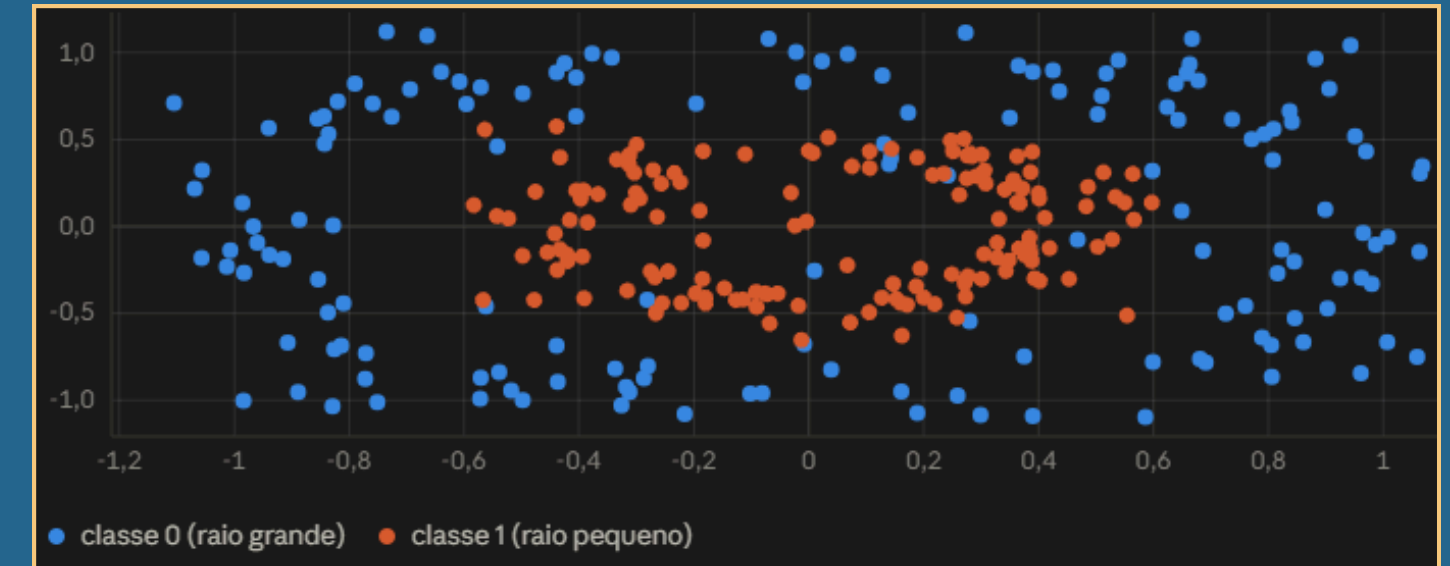
COMO FOI FEITO?



DUAS LUAS - CURVAS SENOIDAIS

- O Perceptron é posto para ajustar as retas senoidais geradas pelo RFF, combinando-as alterando-as com pesos e deslocamentos sorteados uma vez

```
for W, b in zip(self.Ws, self.bs):
    partes.append(math.sqrt(2 / W.shape[1]) * torch.cos(aug @ W + b))
return torch.cat(partes, dim=1)
```

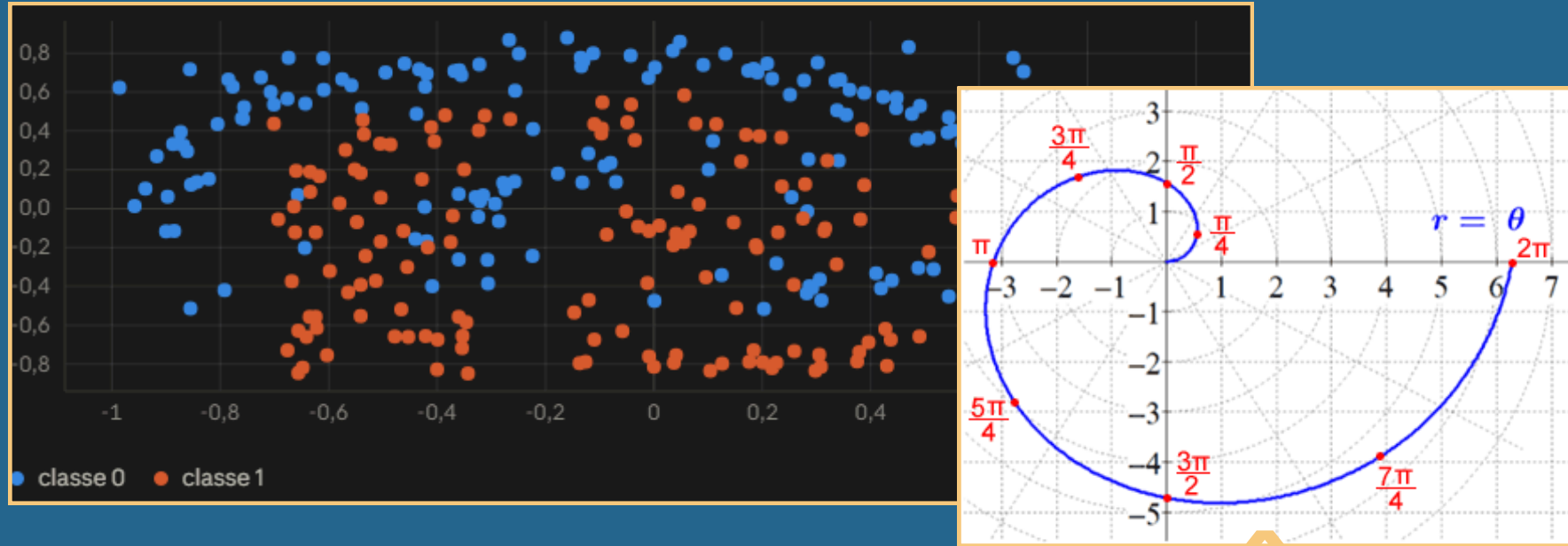


CÍRCULOS

- Dados que x^2 e y^2 já foram calculados, o Perceptron deve aprender a somá-los (com um peso de ajuste) pra gerar a reta que mais se ajusta ao problema.

```
def phi(self, X: torch.Tensor) -> torch.Tensor:
    Xs = (X - self.mu) / self.sd
    partes = [Xs, Xs ** 2]
    if self.pares:
```

COMO FOI FEITO?



ESPIRAL - X E Y EM RAIOS E ÂNGULO

- Transforma x e y em coordenadas polares e as coloca como entradas para RFF
- Ângulo cresce junto com o raio (faz mais sentido se pensar na Espiral de Arquimedes)

```
def _aug(self, Xs: torch.Tensor) -> torch.Tensor:
    if not self.d2:
        return Xs
    r = torch.linalg.norm(Xs, dim=1, keepdim=True)
    theta = torch.atan2(Xs[:, 1], Xs[:, 0]).unsqueeze(1)
    if not hasattr(self, "r_mu"):
        self.r_mu, self.r_sd = r.mean(), r.std() + 1e-8
    return torch.cat([Xs, (r - self.r_mu) / self.r_sd, theta / math.pi], dim=1)
```

ESFERA 10D

- Quadrados ($x^2, \dots$) generalizam pra qualquer dimensão
- A dimensão do problema foi levada em consideração na maior parte da implementação

```
def fit(self, X: torch.Tensor) -> None:
    self.mu, self.sd = X.mean(0), X.std(0) + 1e-8
    d = X.shape[1]
    Xs = (X - self.mu) / self.sd
```

```
pares = [(i, j) for i in range(d) for j in range(i + 1, d)]
explicit_dim = 2 * d + len(pares)
if explicit_dim > self.DIM_MAX - 1:
    pares = []
    explicit_dim = 2 * d
self.pares = pares
self.d2 = (d == 2)
```

```
def _aug(self, Xs: torch.Tensor) -> torch.Tensor:
    if not self.d2:
        return Xs
    r = torch.linalg.norm(Xs, dim=1, keepdim=True)
    theta = torch.atan2(Xs[:, 1], Xs[:, 0]).unsqueeze(1)
    if not hasattr(self, "r_mu"):
        self.r_mu, self.r_sd = r.mean(), r.std() + 1e-8
    return torch.cat([Xs, (r - self.r_mu) / self.r_sd, theta / math.pi], dim=1)
```

**MUITO OBRIGADO
PELA ATENÇÃO!
DÚVIDAS?**

Ok

